Stage 4 Step-by-Step Practical & Conceptual Recap

1. Code Base Expansion: PUT & DELETE Handlers

- **What You Built:** Updated server.js to implement record updating (PUT /tasks/:id) and deletion (DELETE /tasks/:id).
- **Validation & Guard Clauses:**
  - **PUT /tasks/:id**: Checks if the task exists (404 Not Found), checks if at least one valid field (title or done) is provided (400 Bad Request), validates types (non-empty string for title, boolean for done), updates the object in RAM, and returns 200 OK with the updated JSON payload.
  - **DELETE /tasks/:id**: Finds the array index using .findIndex(), removes the record using .splice(index, 1), and returns 204 No Content.
- **Sysadmin Analogy:**
  - PUT is equivalent to running Set-Service or modifying Windows Registry key attributes for an existing service.
  - DELETE is equivalent to unregistering a service daemon or removing an Access Control List (ACL) entry using Remove-Item.

2. Daemon Process Reloading

- **What You Executed:** In your **left terminal tab**, you interrupted the old background process using Ctrl + C and re-executed:
  node server.js

- **Observed Output:**
  Task API Daemon running on http://localhost:3000

- **Sysadmin Analogy:** Stopping and restarting the process flushes outdated bytecode from active volatile RAM and loads the newly saved server.js script into memory, identical to executing systemctl restart myapp.service.

3. Step-by-Step CRUD Protocol Execution

In your **right terminal tab**, you executed the complete lifecycle test suite using curl.exe:

A. Step 1: Instantiate Record (POST /tasks)

- **Command:**
  curl.exe -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Buy milk"}'

- **Result:** HTTP/1.1 201 Created with dynamic primary key assignment {"id":4,"title":"Buy milk","done":false}.

B. Step 2: Update Task Parameters (PUT /tasks/4)

- **Command:**
  curl.exe -i -X PUT http://localhost:3000/tasks/4 -H "Content-Type: application/json" -d '{"title":"Buy fresh organic milk"}'

- **Result:** HTTP/1.1 200 OK returning the updated object payload with the new title.

C. Step 3: Mutate State / Mark Completed (PUT /tasks/4)

- **Command:**
  curl.exe -i -X PUT http://localhost:3000/tasks/4 -H "Content-Type: application/json" -d '{"done":true}'

- **Result:** HTTP/1.1 200 OK returning {"id":4,"title":"Buy fresh organic milk","done":true}.

D. Step 4: Purge Record (DELETE /tasks/4)

- **Command:**
  curl.exe -i -X DELETE http://localhost:3000/tasks/4

- **Result:** HTTP/1.1 204 No Content (Empty payload body returned as per standard HTTP spec).
- **Sysadmin Analogy:** HTTP 204 No Content functions like a silent command completion in Linux or PowerShell—the action succeeded cleanly, so no diagnostic text body is required.

4. State & Error Gate Verification

- **Verifying Purge from Inventory (GET /tasks):**
  curl.exe -i http://localhost:3000/tasks

  Confirmed task id: 4 was permanently removed from active memory array.
- **Verifying 404 Firewall Response (GET /tasks/4):**
  curl.exe -i http://localhost:3000/tasks/4

  - **Observed Response:**
    HTTP/1.1 404 Not Found
    Content-Type: application/json; charset=utf-8

    {"error":"Task 4 not found"}

  - **Verification Outcome:** Confirmed that attempting to query a purged record correctly triggers the 404 guard clause.

5. Version Control Checkpoint & Commit

- **What You Executed:** Saved server.js (Ctrl + S), then staged and committed the full working repository state:
  git add .
  git commit -m "Stage 4: full CRUD"

- **Observed Commit Audit Output:**
  [master 6f95409] Stage 4: full CRUD
  6 files changed, 60 insertions(+), 4 deletions(-)

- **Sysadmin Analogy:** The commit 6f95409 establishes an immutable system restore point, confirming your complete REST API build milestone in Git history.